

«Почему Docker Compose - это не Production»

Docker Compose был создан как инструмент для локальной разработки и простого запуска нескольких контейнеров. Он отлично подходит для тестирования, обучения и небольших окружений, но при попытке использовать его в production проявляются фундаментальные ограничения.

1. Отсутствие Self-healing

В production-среде отказоустойчивость является критически важным требованием. Docker Compose не отслеживает фактическое состояние приложения. Если контейнер упал из-за ошибки, утечки памяти или внешнего сбоя, Compose не принимает решений на уровне системы.

Механизм restart: always не является self-healing:

- он не проверяет, *почему* контейнер упал
- он не анализирует health checks
- он не перераспределяет нагрузку

Таким образом, Compose не гарантирует, что сервис будет постоянно доступен.

2. Ограниченное Scaling

Docker Compose позволяет запускать несколько экземпляров сервиса, но:

- не предоставляет встроенной балансировки нагрузки
- не знает, сколько экземпляров реально нужно
- не умеет масштабироваться динамически

Масштабирование в Compose:

- ручное
- статичное
- не реактивное

В production же требуется автоматическое масштабирование в зависимости от нагрузки, чего Compose принципиально не поддерживает.

3. Отсутствие Zero-downtime deployment

Обновление сервисов в Docker Compose приводит к остановке контейнеров.

Даже при использовании нескольких экземпляров нет механизма:

- постепенной замены контейнеров
- контроля готовности новых версий
- отката в случае ошибки

Любое обновление влечёт за собой downtime, что недопустимо для production-систем.

Итог

Docker Compose:

- не обеспечивает отказоустойчивость
- не управляет масштабированием
- не поддерживает бесшовные обновления

Он решает задачу **запуска контейнеров**, но не задачу **управления сервисами**.

Поэтому Docker Compose - это инструмент разработки и обучения, но **не production-оркестратор**.